Fase 5 - Observabilidad con Prometheus, Grafana y cAdvisor

Objetivo

Incorporar capacidades de monitoreo y observabilidad al laboratorio DevOps mediante Prometheus, Grafana, Node Exporter y cAdvisor, permitiendo monitorear infraestructura, contenedores Docker y la aplicacion Flask.

Arquitectura inicial

La fase comenzo con una arquitectura compuesta por Nginx como Reverse Proxy y Flask como aplicacion principal ejecutados sobre Docker Compose.

Implementacion de Prometheus

Se creo el stack de monitoreo utilizando Docker Compose. Se desplego Prometheus como repositorio central de metricas y se configuraron los targets iniciales para recolectar informacion.

Implementacion de Node Exporter

Se incorporo Node Exporter para exponer metricas del sistema operativo Ubuntu, incluyendo CPU, memoria, disco, filesystem, red y carga del sistema.

Implementacion de Grafana

Se desplego Grafana mediante Docker, se realizo la configuracion inicial, el cambio de credenciales por defecto y la integracion con Prometheus mediante un Data Source dedicado.

Dashboards de infraestructura

Se importo el dashboard Node Exporter Full para visualizar metricas de CPU, memoria, disco, red y estado general del servidor Ubuntu.

Implementacion de cAdvisor

Se agrego cAdvisor al stack para monitorear contenedores Docker individualmente. Se configuraron los volumenes necesarios para acceder a informacion del host y de Docker.

Monitoreo de contenedores

Se importo un dashboard especifico para Docker obteniendo consumo de CPU, RAM, red y almacenamiento de Flask, Nginx, Prometheus, Grafana y cAdvisor.

Instrumentacion de Flask

Se incorporo la libreria prometheus-client al proyecto Flask. Se creo el endpoint /metrics y se instrumento la aplicacion para exponer metricas personalizadas.

Primera metrica personalizada

Se implemento flask_requests_total utilizando un Counter de Prometheus para contabilizar la cantidad total de solicitudes recibidas por la aplicacion.

Integracion con Prometheus

Se agrego Flask como target de Prometheus y se verifico el correcto funcionamiento ejecutando consultas PromQL sobre la metrica flask_requests_total.

Problema encontrado - Fecha y hora

Luego de suspender el equipo host, la maquina virtual quedo con la hora desactualizada. Esto genero inconvenientes potenciales para sincronizacion, metricas y servicios dependientes del tiempo.

Diagnostico realizado

Se utilizaron comandos como timedatectl, chronyc tracking y chronyc sources para verificar el estado de sincronizacion horaria y analizar el comportamiento de Chrony.

Resolucion

Se corrigio la hora utilizando Chrony, se verifico la sincronizacion del reloj y se comprendio la diferencia entre Save State, Suspension y apagado correcto de una VM.

Conceptos aprendidos

Observabilidad, Prometheus, Grafana, Exporters, Targets, Scraping, Instrumentacion, PromQL, Dashboards, Monitoreo de Infraestructura, Monitoreo de Contenedores y Monitoreo de Aplicaciones.

Competencias adquiridas

Prometheus, Grafana, Node Exporter, cAdvisor, PromQL, Docker Monitoring, Application Monitoring, Instrumentacion Python, Analisis de metricas, Troubleshooting de NTP y Observabilidad moderna.

Resultado final

El laboratorio quedo equipado con una plataforma completa de monitoreo capaz de observar Ubuntu, Docker, Nginx, Flask y los servicios de observabilidad en tiempo real mediante dashboards y metricas centralizadas.